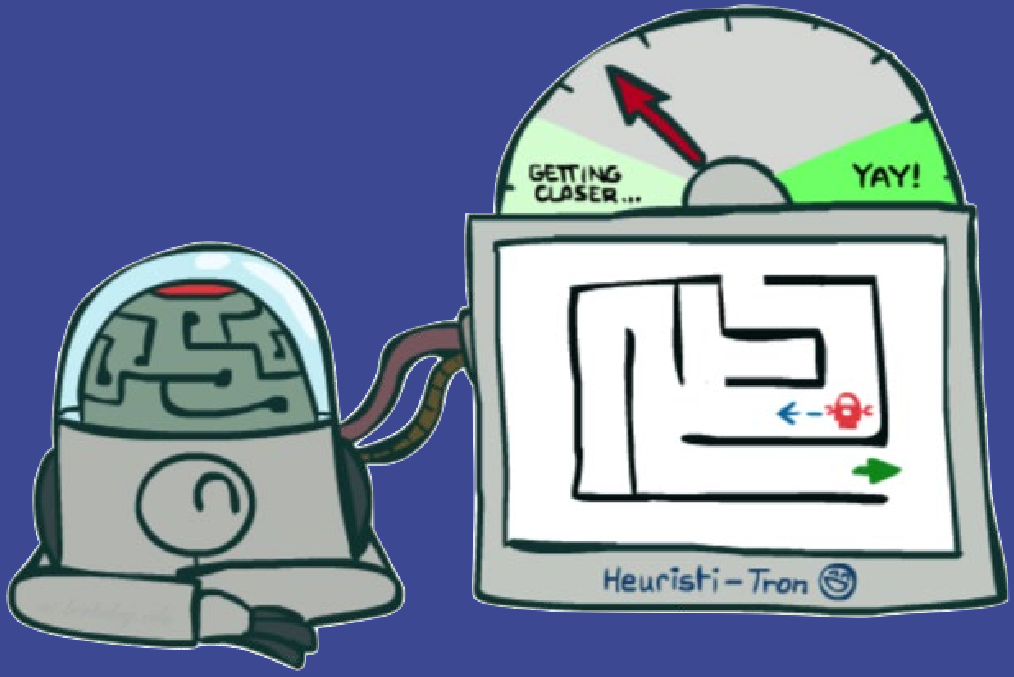
CS 156
Intro to AI

Heuristics



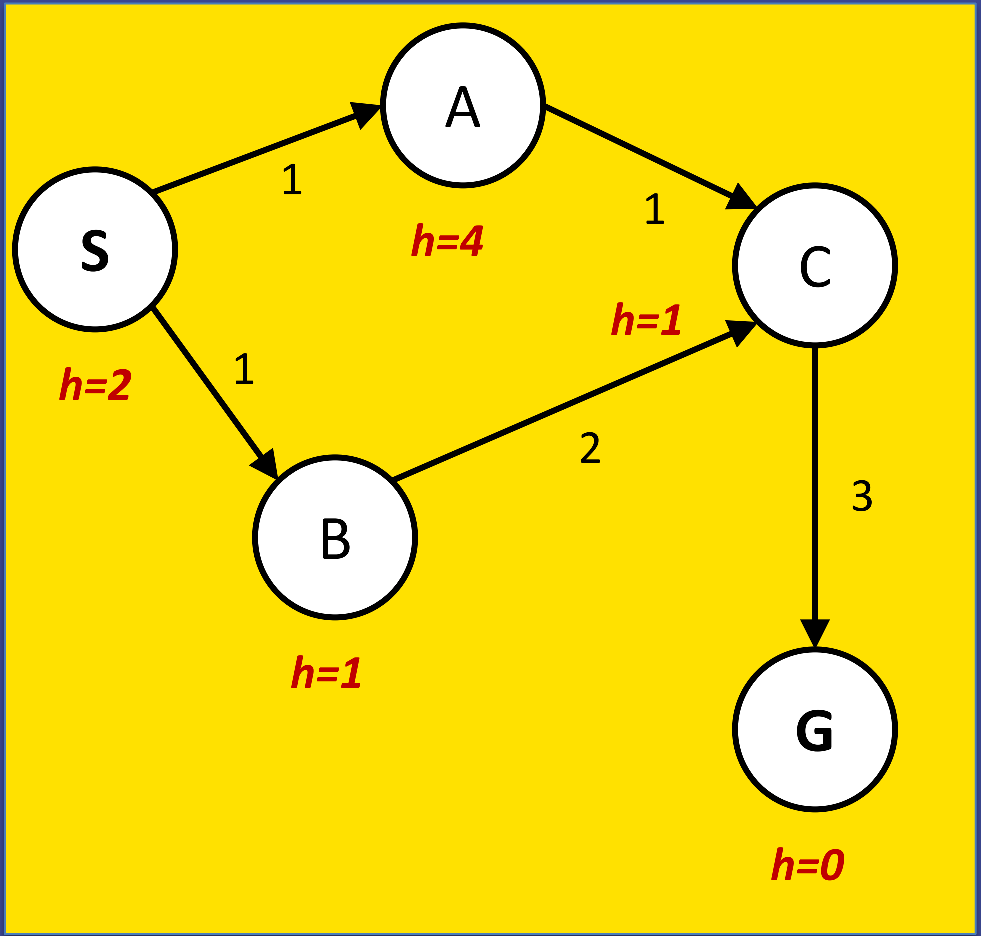
These slides are based on the slides created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley.
The artwork is by Ketrina Yim

Today

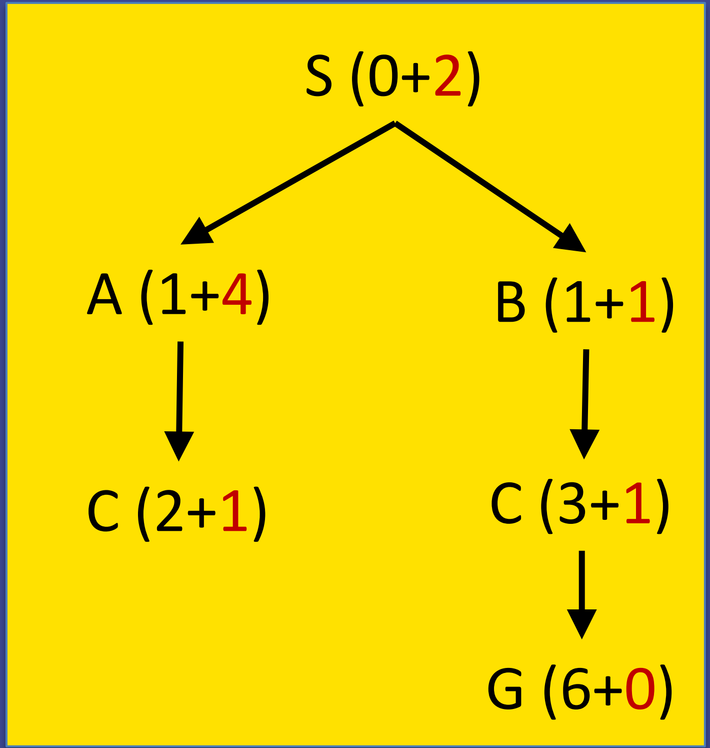
- Consistency
- Properties of A*
- Creating Heuristics
- Properties of Heuristics

A* Graph Search with Admissible Heuristic?

State space graph



Search tree



Closed set

S B C A

SAC is not expanded because C is in closed set.
We end up with a suboptimal path SBCG!

Consistency of Heuristics

Main idea: estimated heuristic costs ≤ actual costs

- Admissibility:

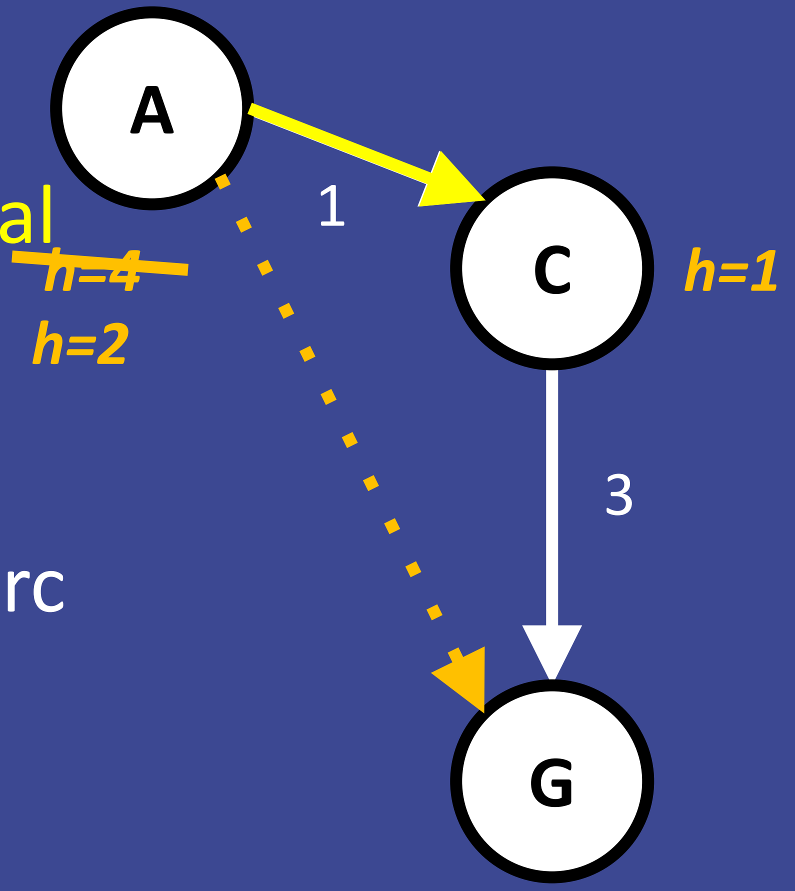
  heuristic cost ≤ actual cost to a nearest goal

  h(A) ≤ actual cost from A to G

- Consistency:

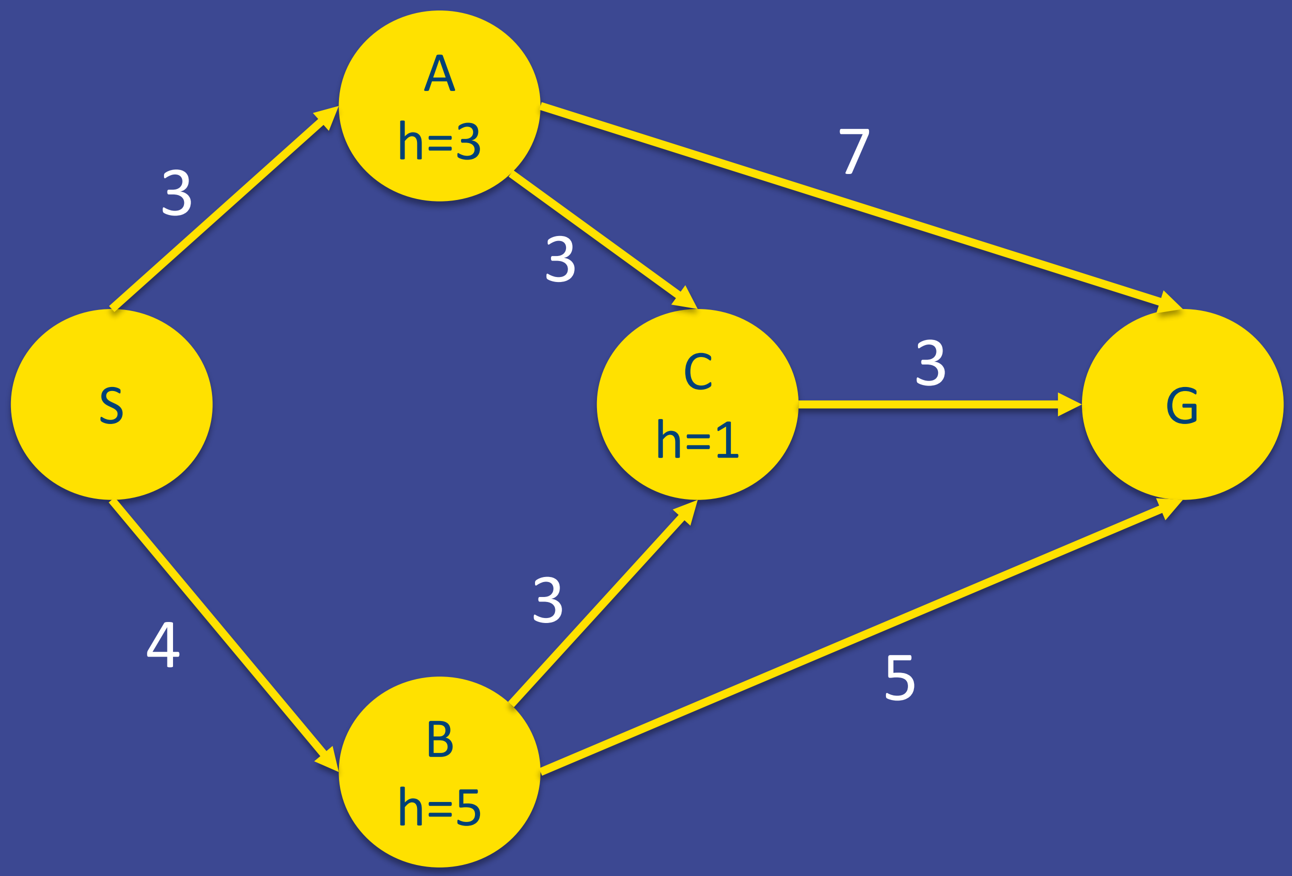
  heuristic "arc" cost ≤ actual cost for each arc

  h(A) – h(C) ≤ cost(A to C)



Consistent Heuristic?

A. Yes
B. No



Consequence of Consistency

For two neighbors, X and Y:

$h(X) - h(Y) \leq cost(X \text{ to } Y)$

$h(X) \leq cost(X \text{ to } Y) + h(Y)$

$cost(S \text{ to } X) + h(X) <= cost(S \text{ to } X) + cost(X \text{ to } Y) + h(Y)$

$f(X) <= f(Y)$

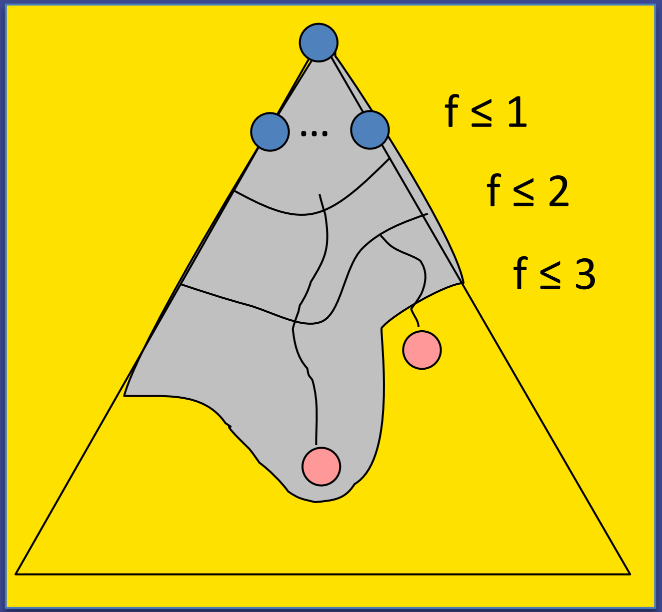
- The *f* value along a path never decreases

Consequence of Consistency

Consider what A* does with a consistent heuristic:

1. A* expands nodes in increasing total f value (f-contours)
2. For every state s, nodes that reach s optimally are expanded before nodes that reach s suboptimally

- A* graph search is optimal



Optimality of A* Search

Tree search:

- A* is optimal if heuristic is admissible
- UCS is a special case (h = 0 is admissible)

Graph search:

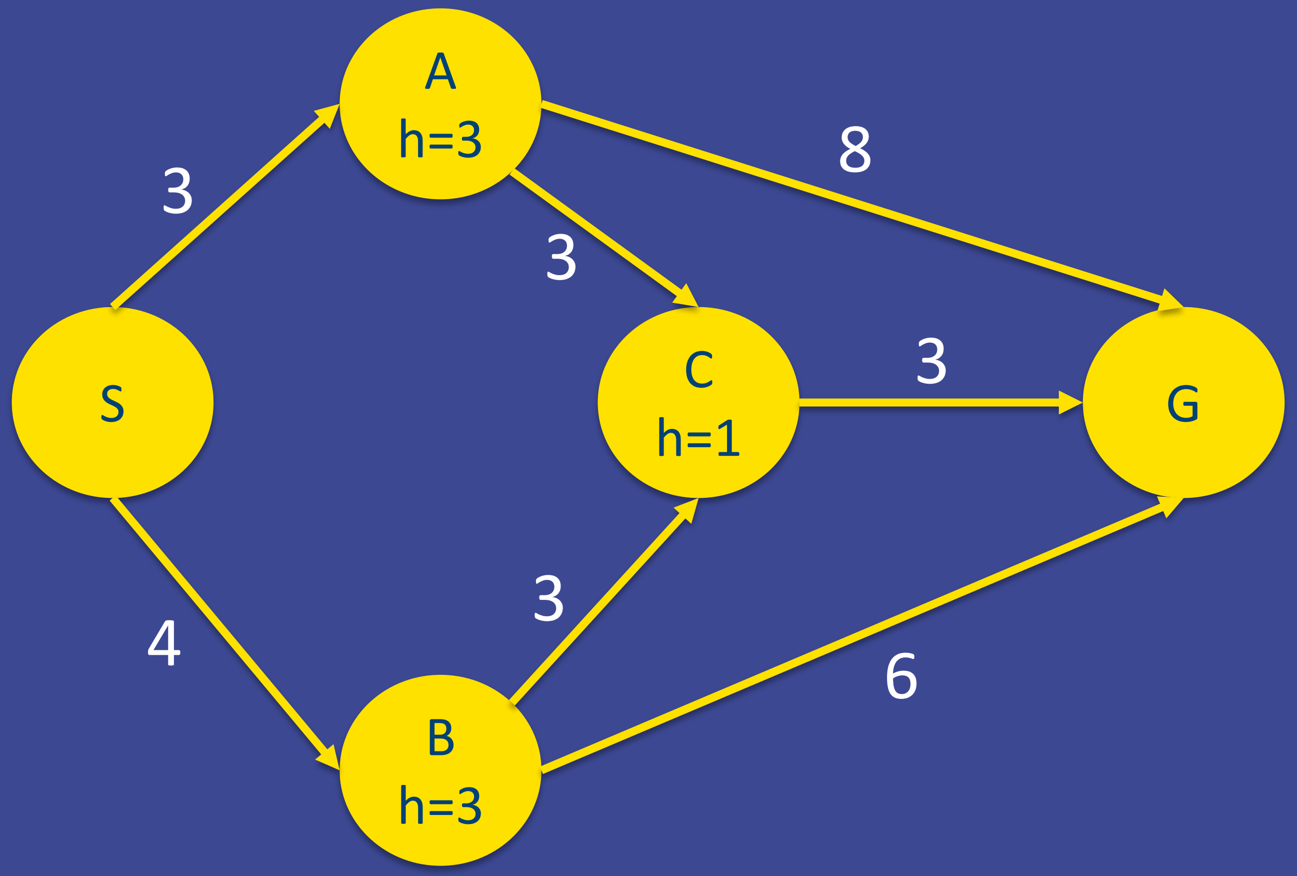
- A* optimal if heuristic is consistent
- UCS is a special case (h = 0 is consistent)

Consistency implies admissibility

In general, most natural admissible heuristics tend to be consistent, especially if they are derived from relaxed problems.

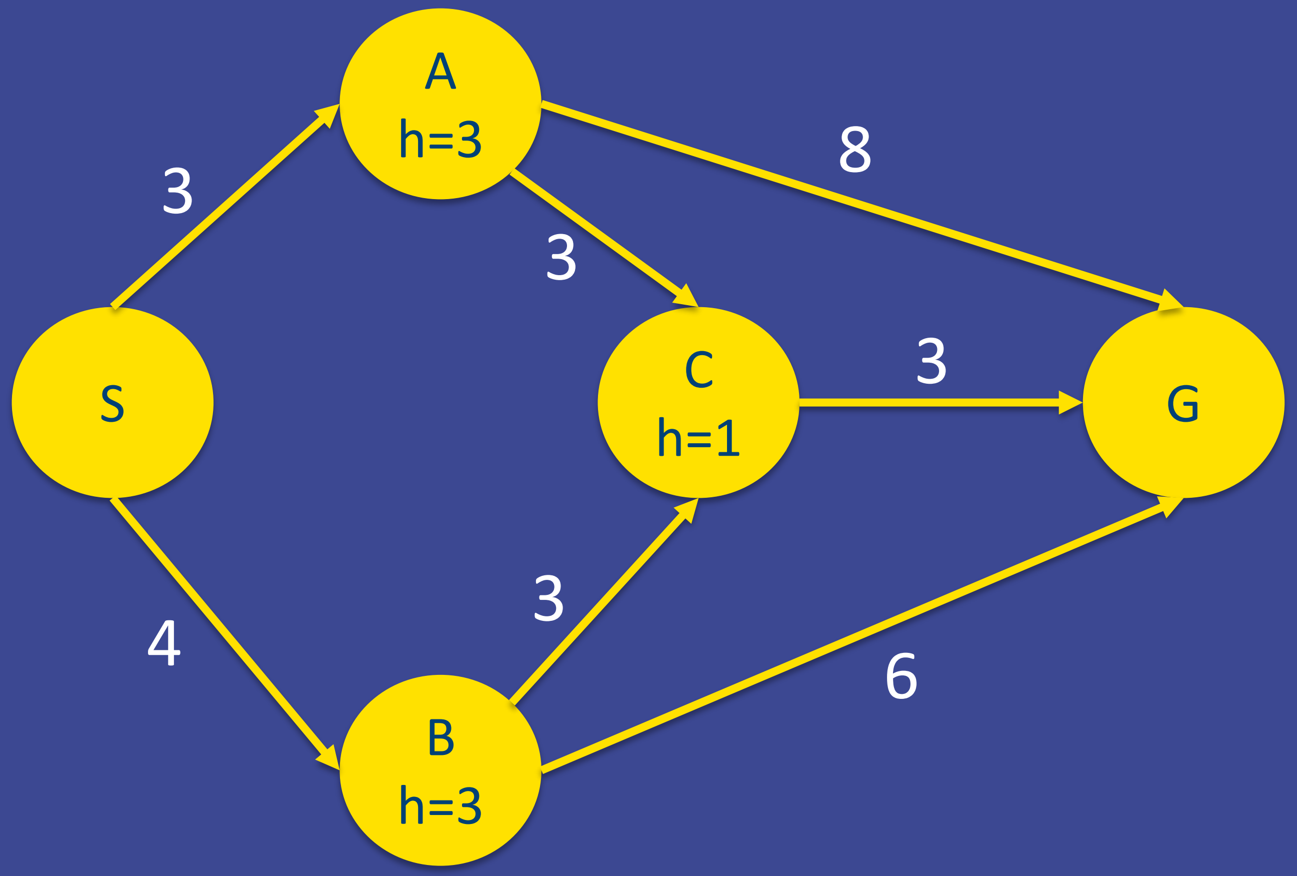
Admissible Heuristic?

A. Yes
B. No

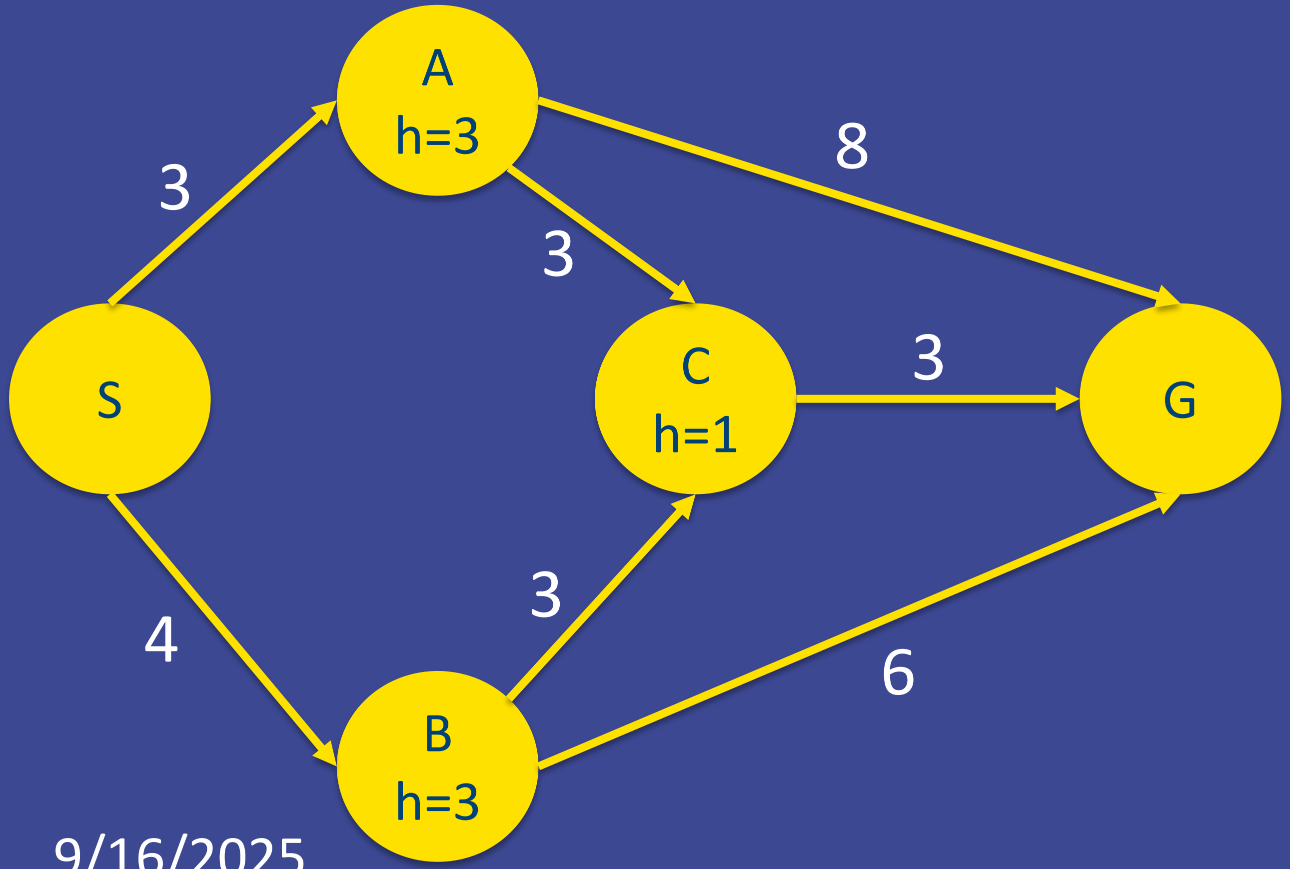


Consistent Heuristic?

A. Yes
B. No



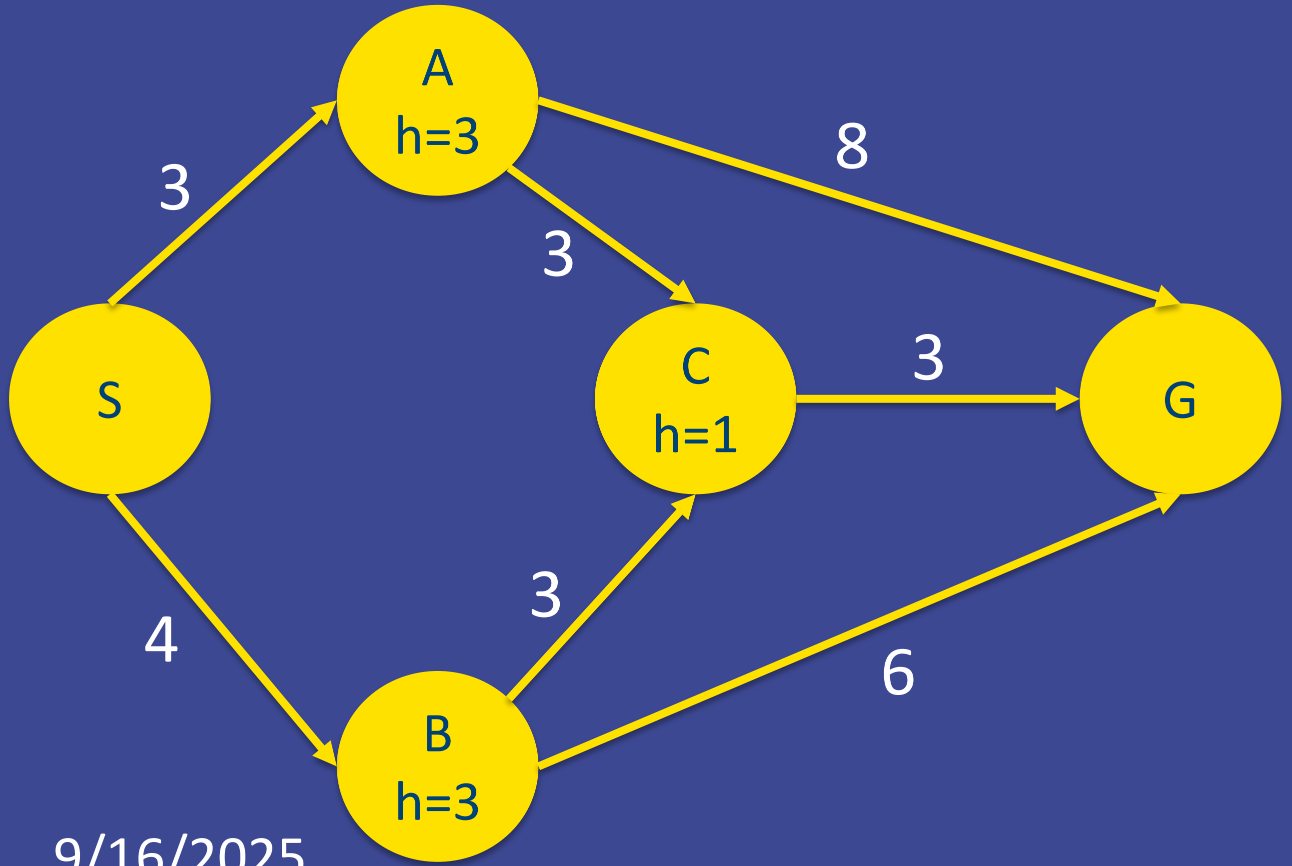
A* Step by Step Expansion



Fringe	g	h	f
S	0	?	?

Closed set:

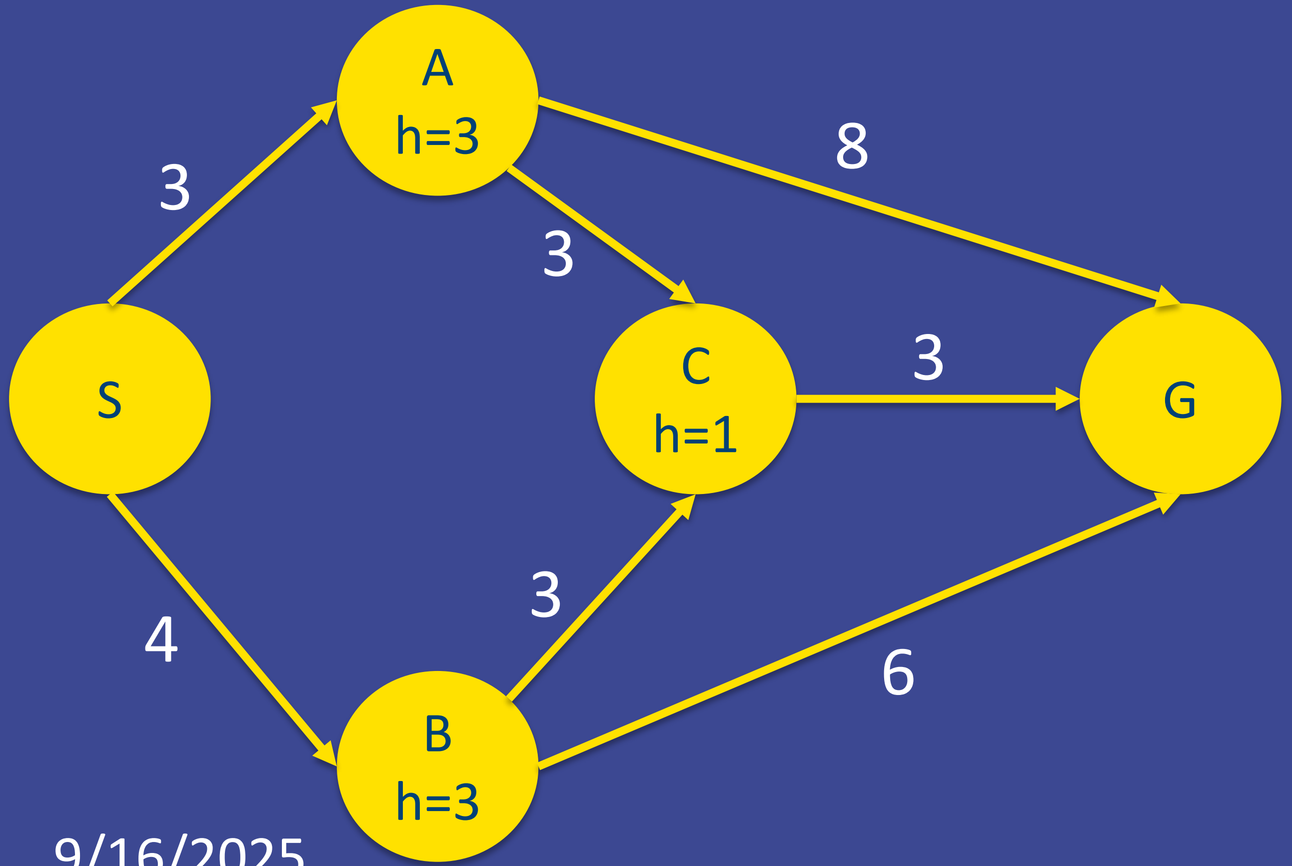
A* Step by Step Expansion



Fringe	g	h	f
~~S~~	0	?	?
SA	3	3	6
SB	4	3	7

Closed set: S

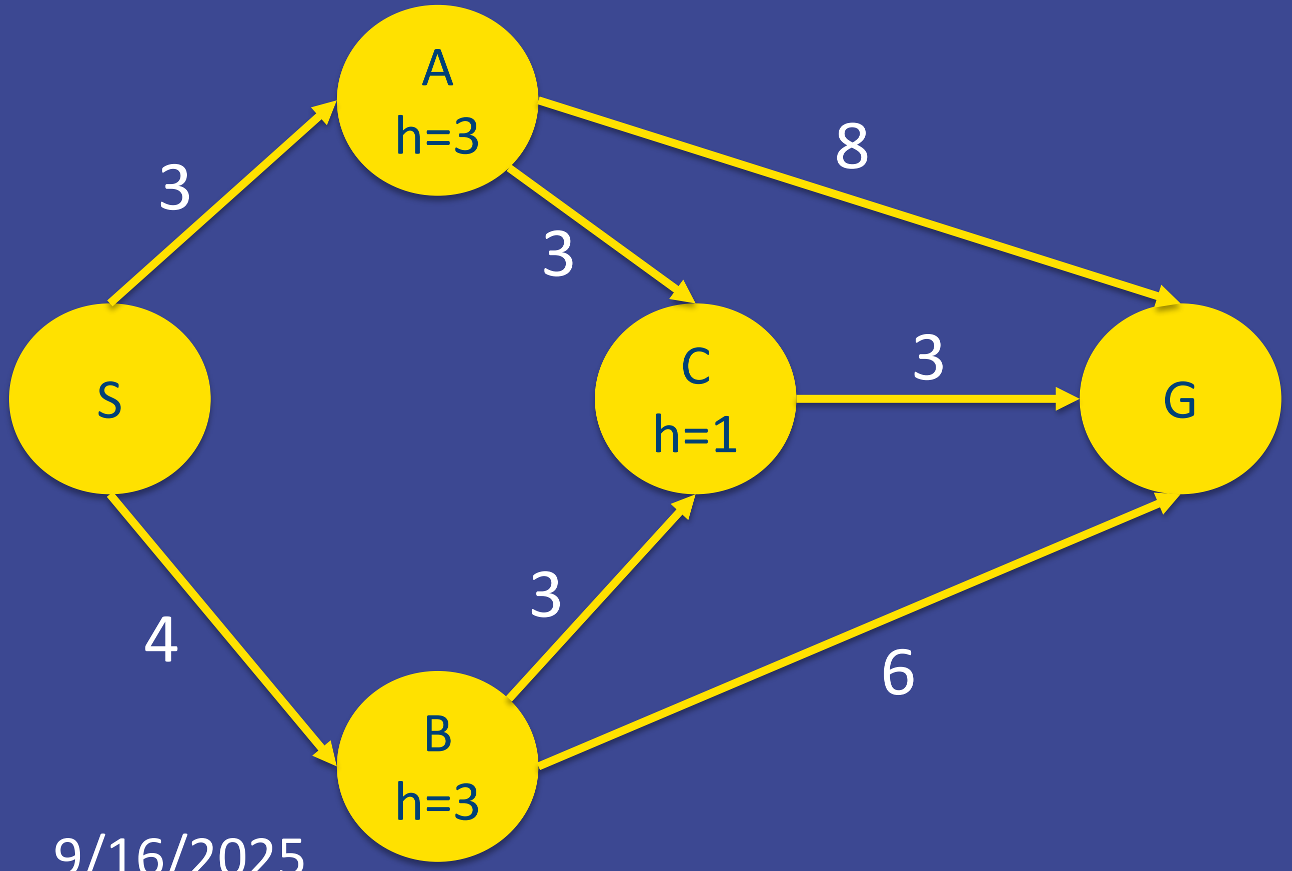
A* Step by Step Expansion



Fringe	g	h	f
~~S~~	0	?	?
~~SA~~	3	3	6
SB	4	3	7
SAC	6	1	7
SAG	11	0	11

Closed set: S, A

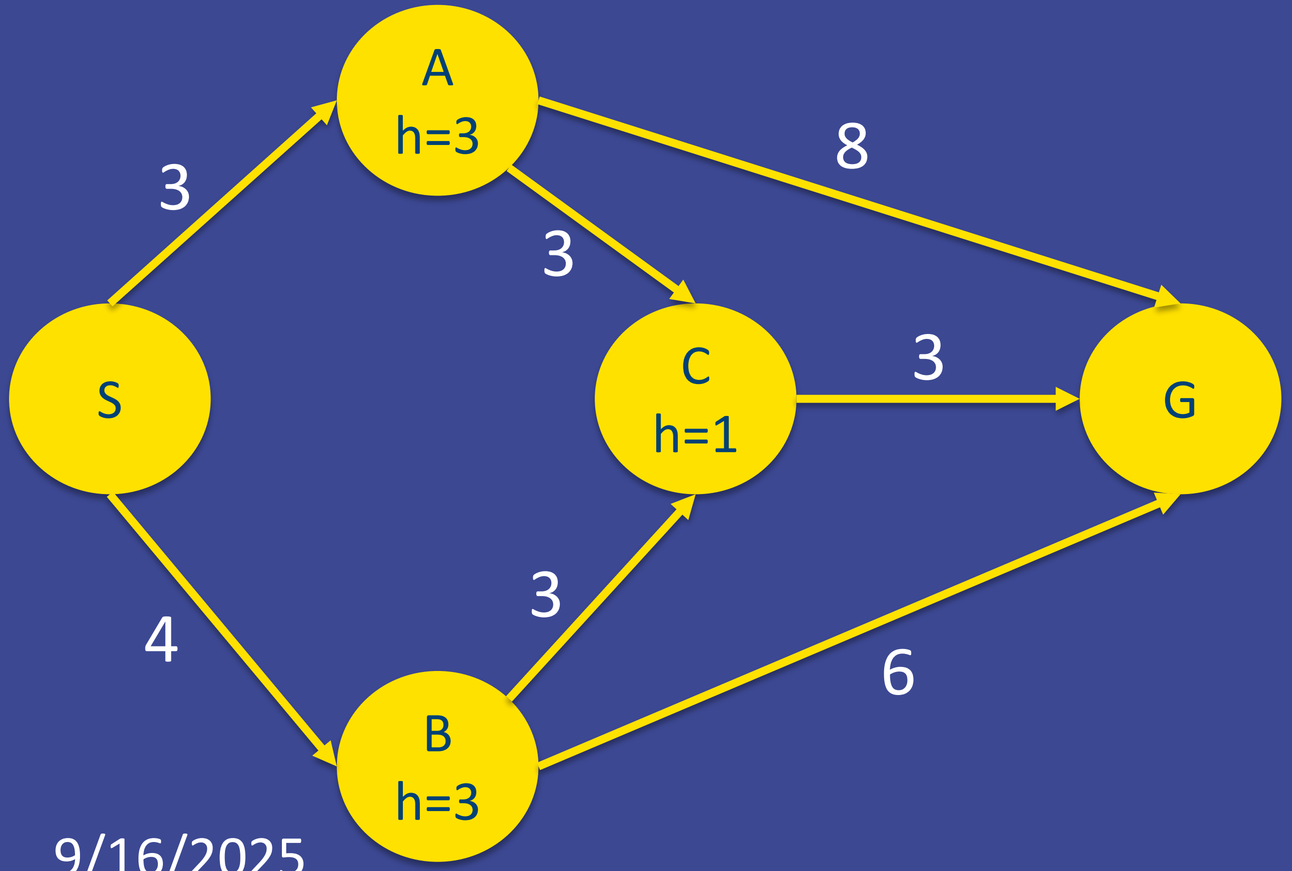
A* Step by Step Expansion



Fringe	g	h	f
~~SB~~	4	3	7
SAC	6	1	7
SAG	11	0	11
SBC	7	1	8
SBG	10	0	10

Closed set: S, A, B

A* Step by Step Expansion



Fringe	g	h	f
~~SAC~~	6	1	7
SAG	11	0	11
SBC	7	1	8
SBG	10	0	10
SACG	9	0	9

Closed set: S, A, B, C

A* Step by Step Expansion

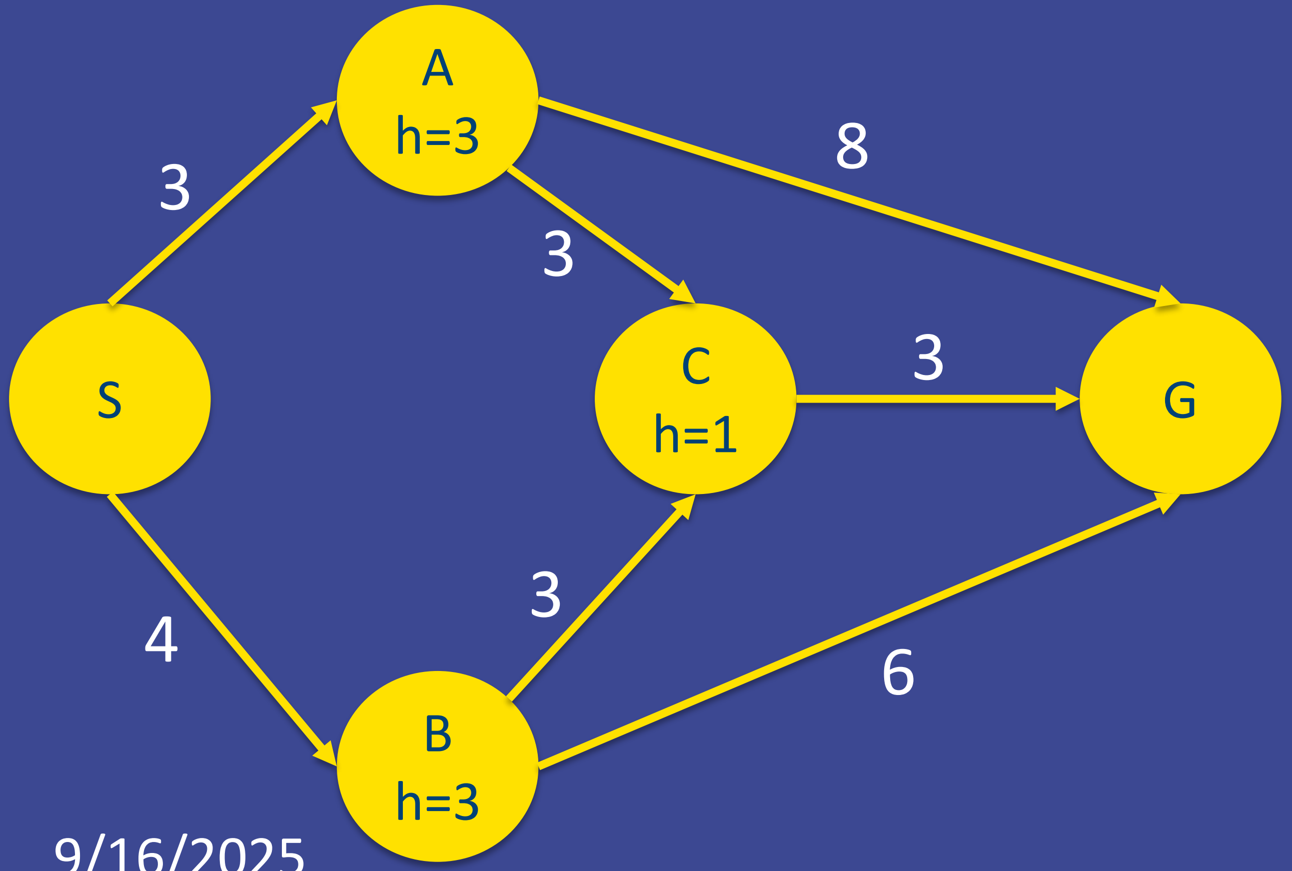
Which node do we expand next?

A. SAG
B. SBC
C. SBG
D. SACG

Fringe	g	h	f
SAG	11	0	11
SBC	7	1	8
SBG	10	0	10
SACG	9	0	9

Closed set: S, A, B, C

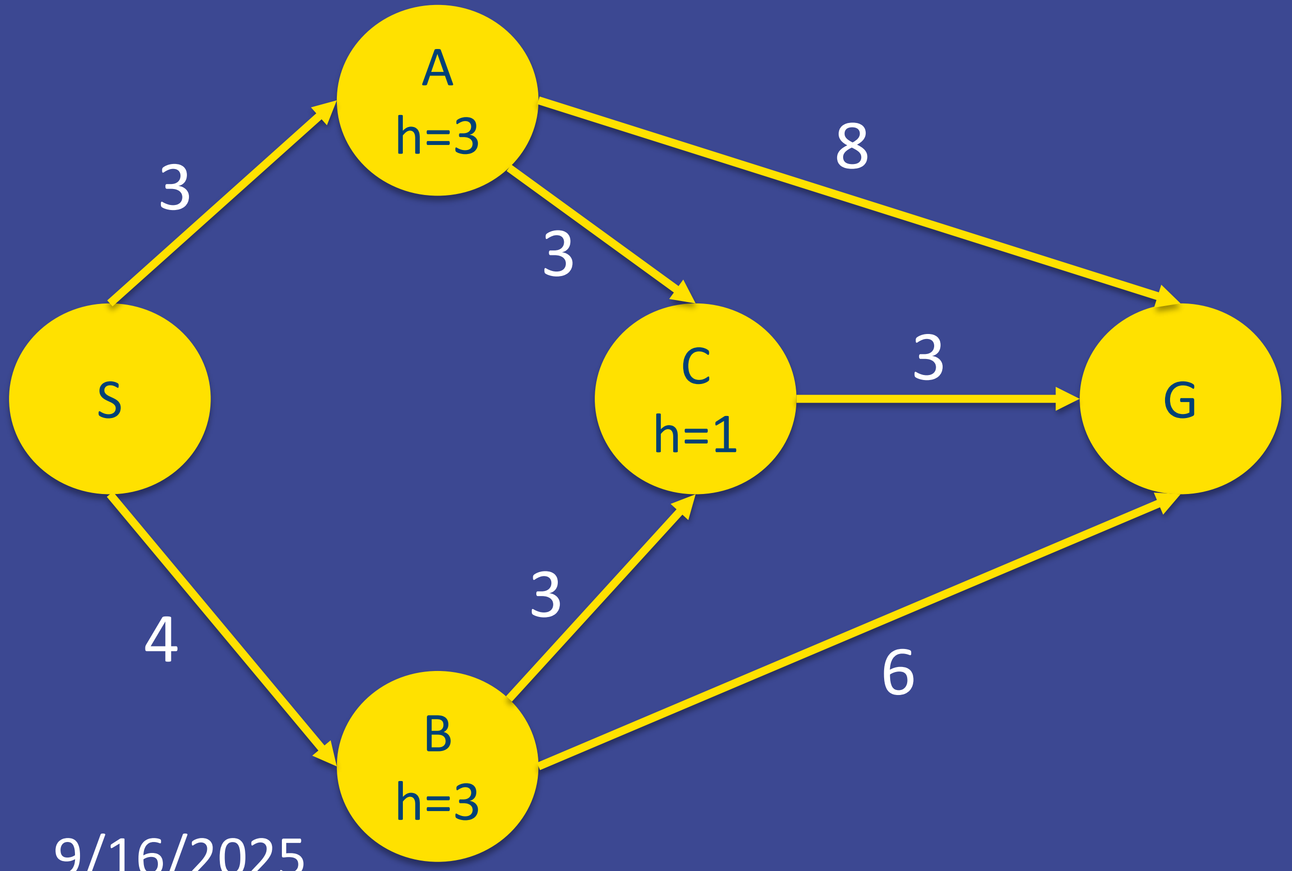
A* Step by Step Expansion



Fringe	g	h	f
SAG	11	0	11
~~SBC~~	~~7~~	~~1~~	~~8~~
SBG	10	0	10
SACG	9	0	9

Closed set: S, A, B, C

A* Step by Step Expansion

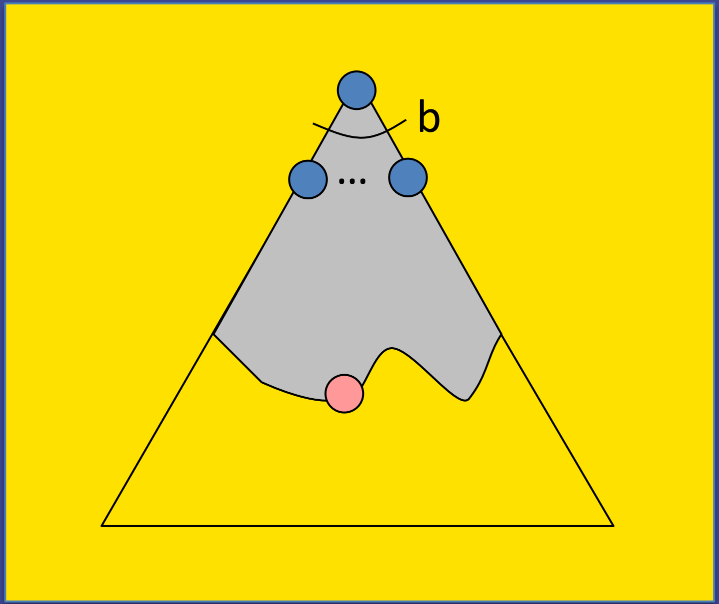


Fringe	g	h	f
SAG	11	0	11
SBG	10	0	10
SACG	9	0	9
SBCG	10	0	10

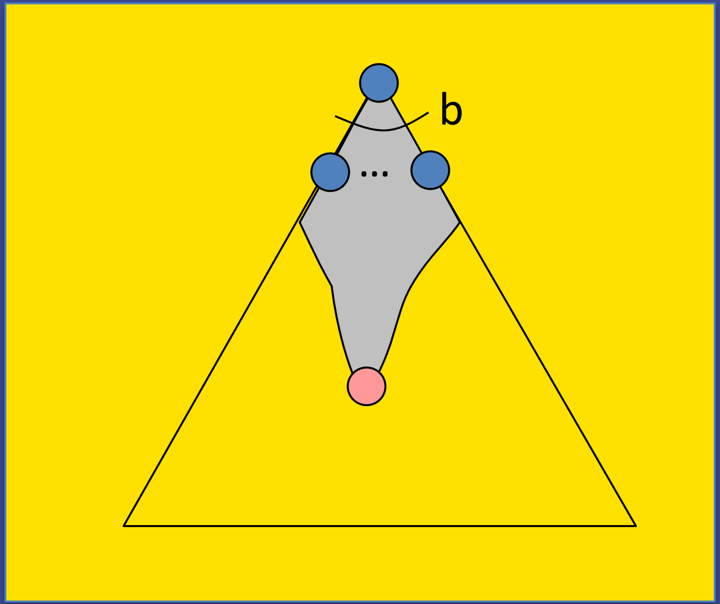
Properties of A*

Properties of A*

Uniform-Cost

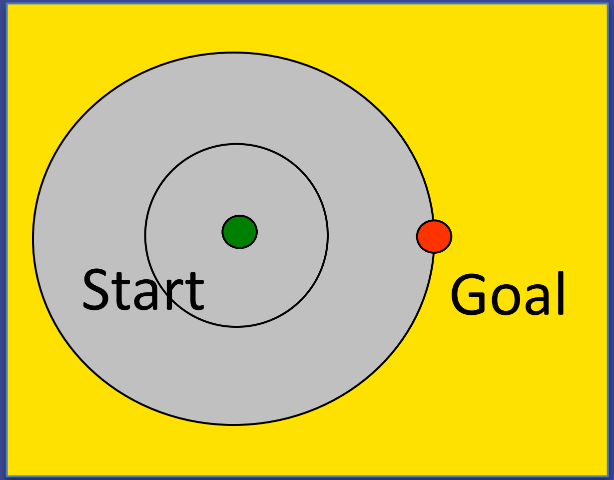


A*

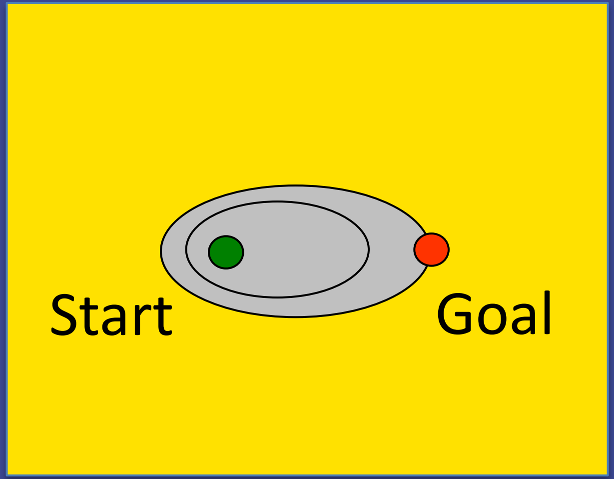


UCS vs A* Contours

- Uniform-cost expands equally in all "directions"



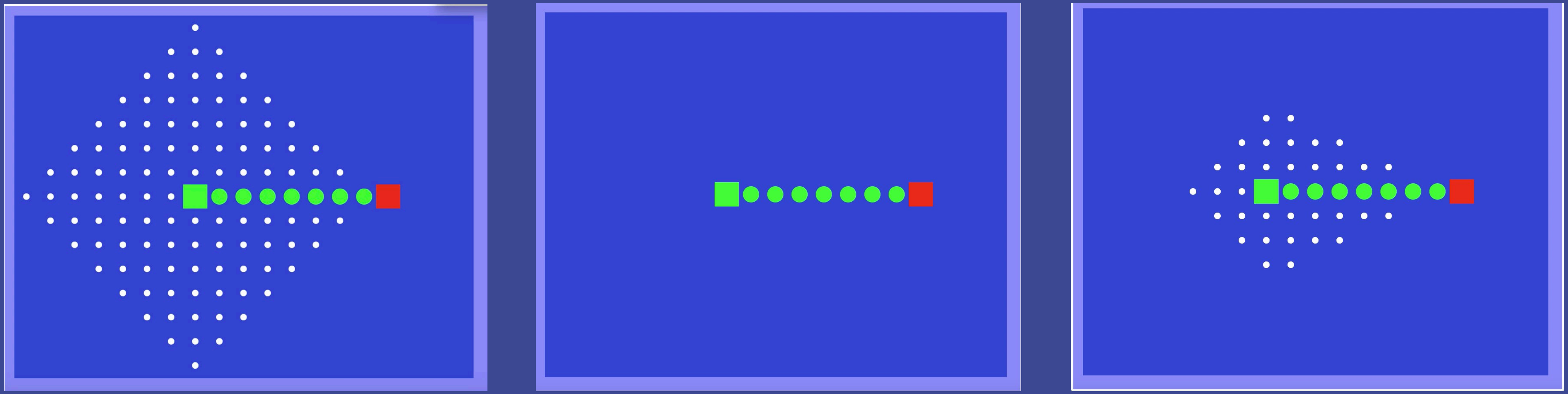
- A* expands mainly toward the goal, but hedges its bets to ensure optimality



Complexity of A*

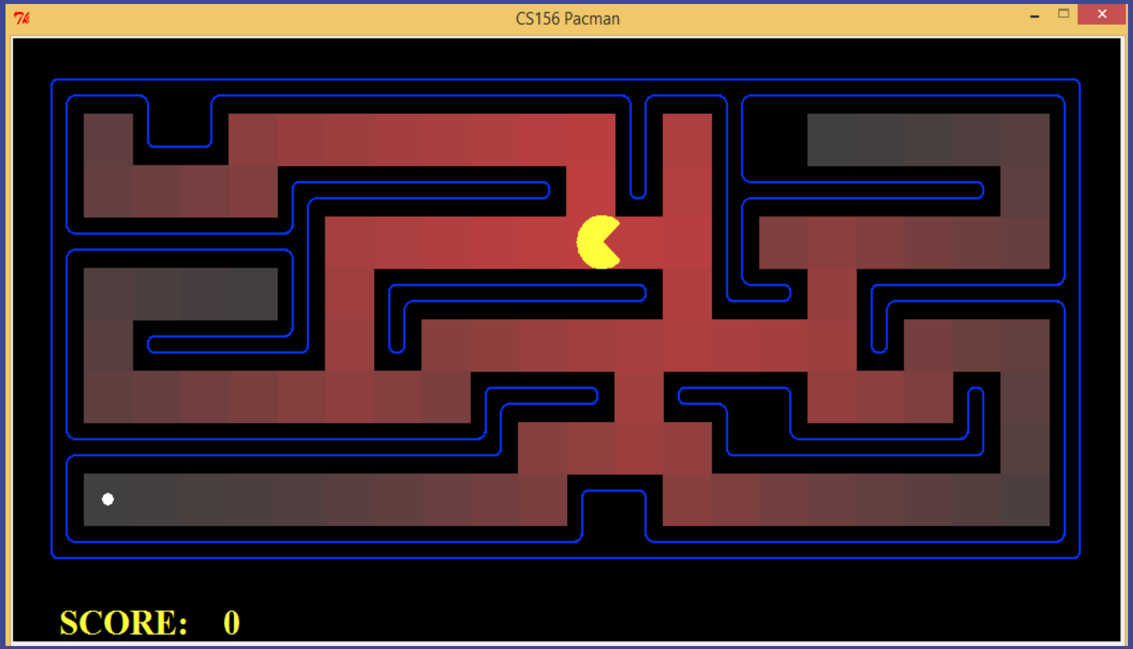
- Depends on the heuristic!
- Worst case (null heuristic) same complexity as UCS
  - time complexity $O(b^{C^*/e})$ (exponential in effective depth)
  - space complexity: $O(b^{C^*/e})$
- In practice, the heuristic reduces the effective branching factor
  - $O(b'^{C^*/e})$
  - b': effective branching factor
  - C^*/e: effective depth

Demo: Contours (Empty)



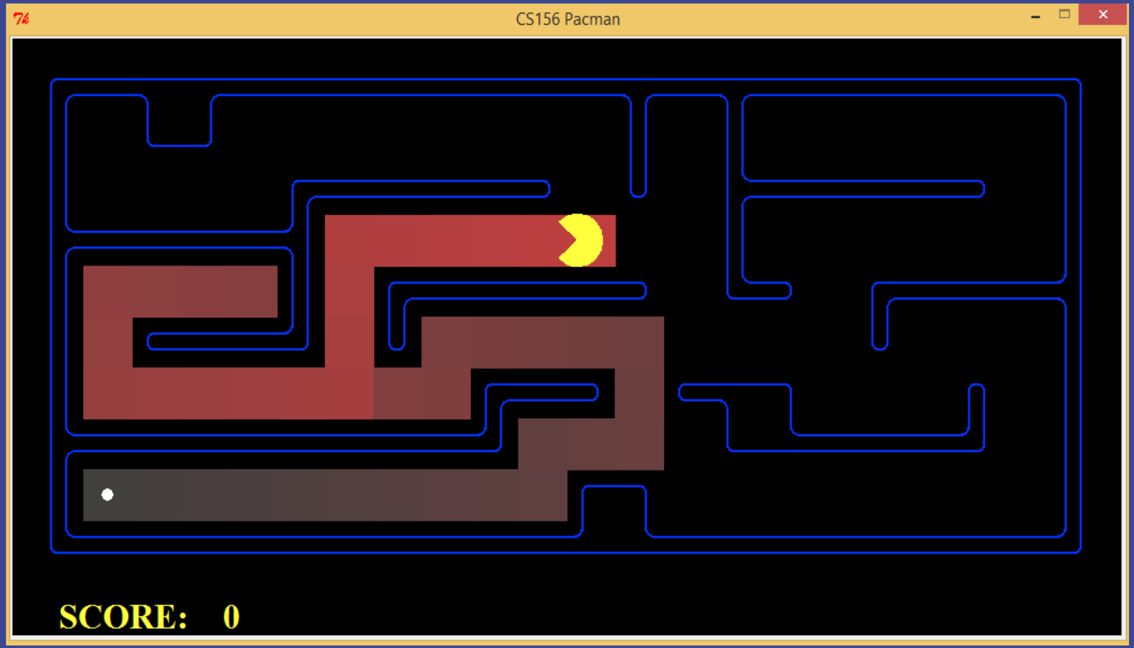
Demo Pacman Small Maze

A. DFS
B. UCS
C. Greedy
D. A*



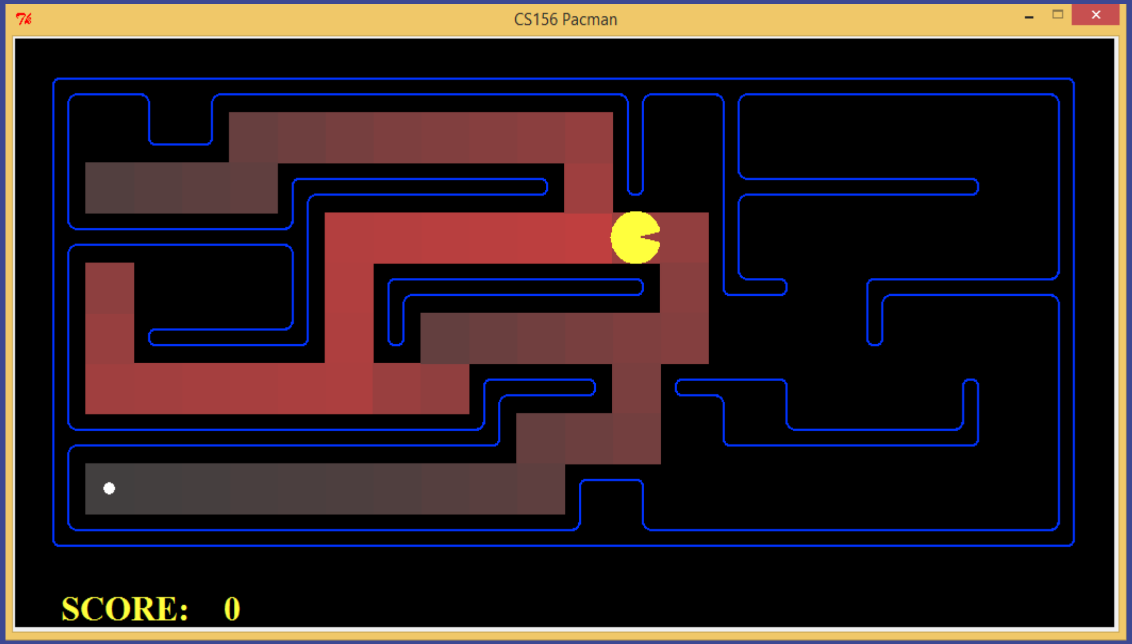
Demo Pacman Small Maze

A. DFS
B. UCS
C. Greedy
D. A*



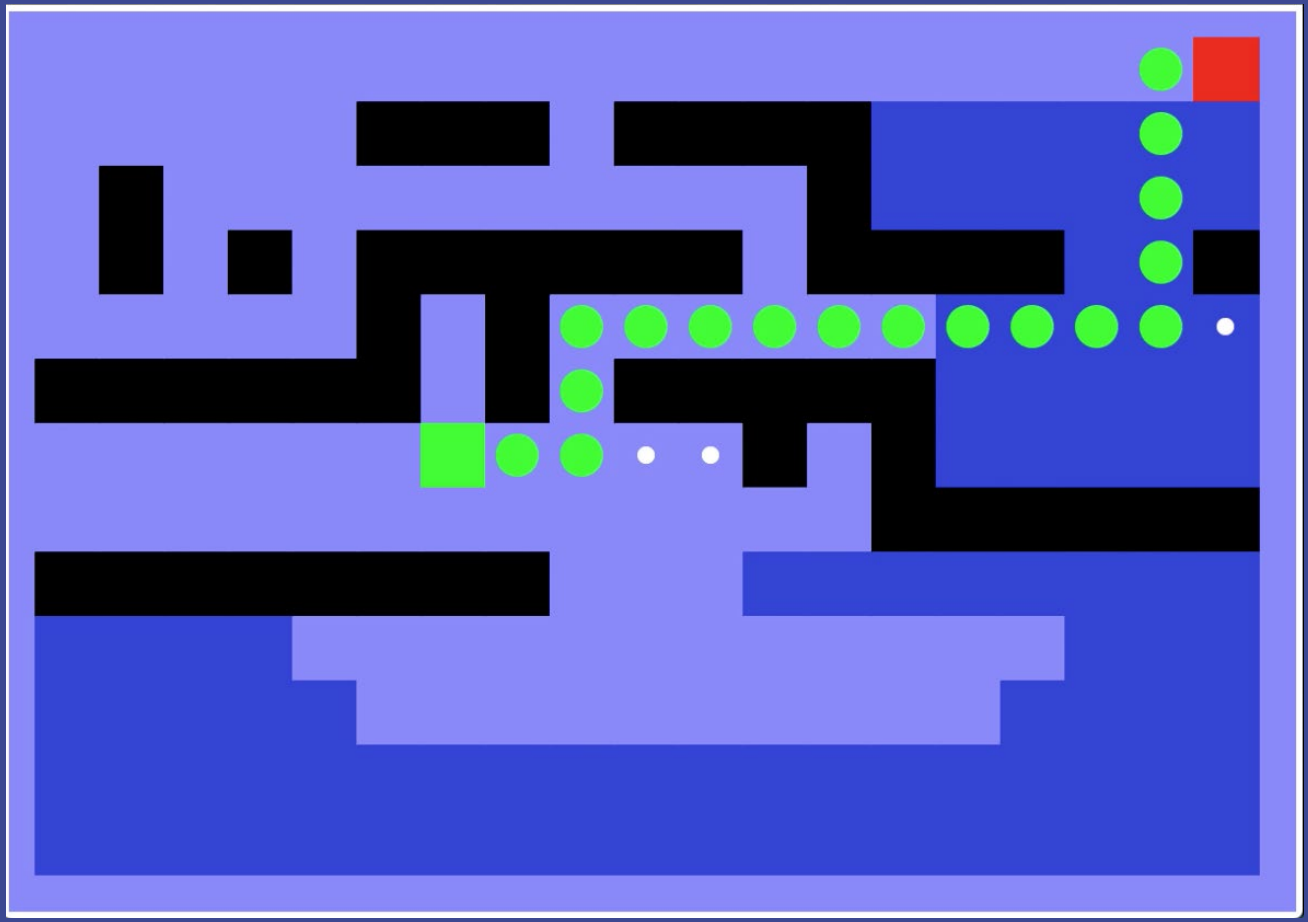
Demo Pacman Small Maze

A. DFS
B. UCS
C. Greedy
D. A*



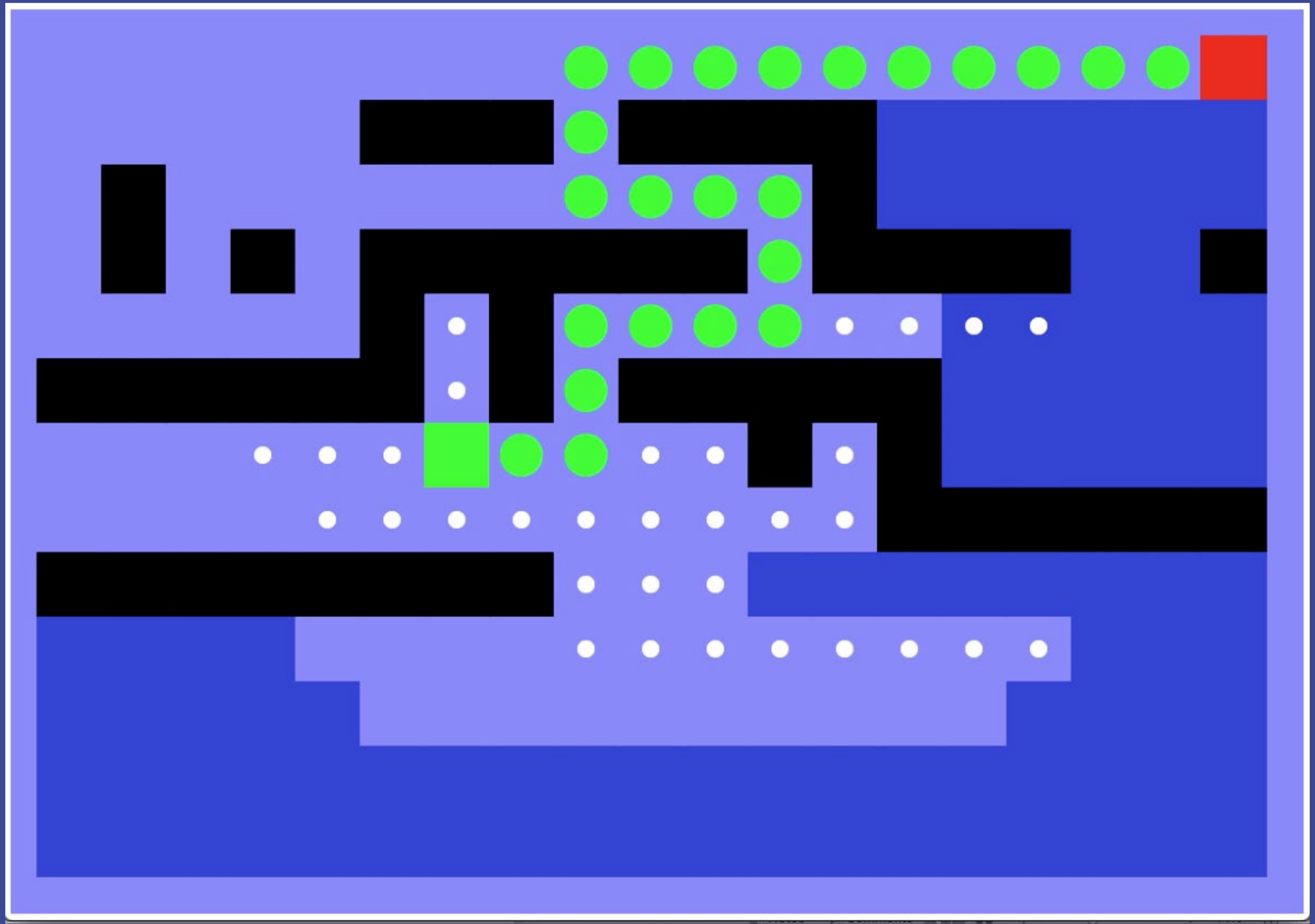
Maze: Deep Shallow Water

A. UCS
B. Greedy
C. A*
D. DFS



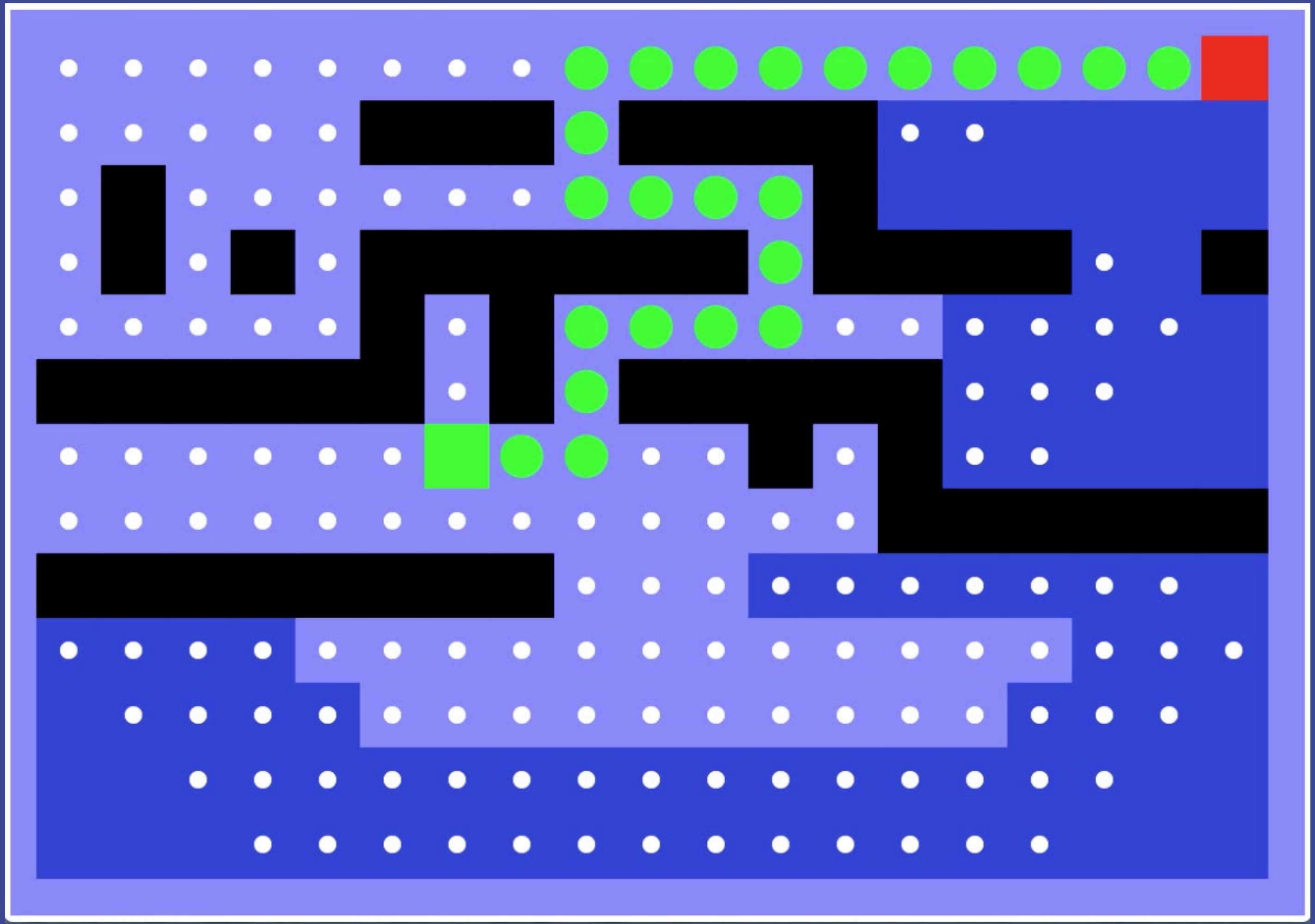
Maze: Deep Shallow Water

A. UCS
B. Greedy
C. A*
D. DFS



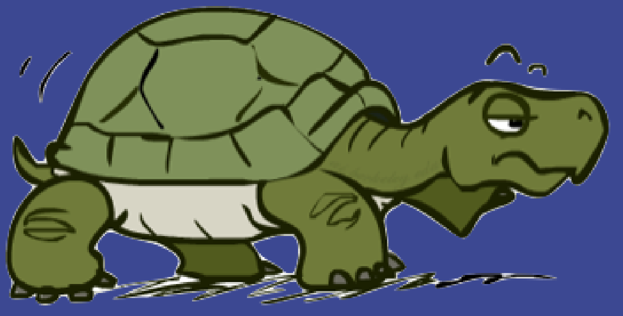
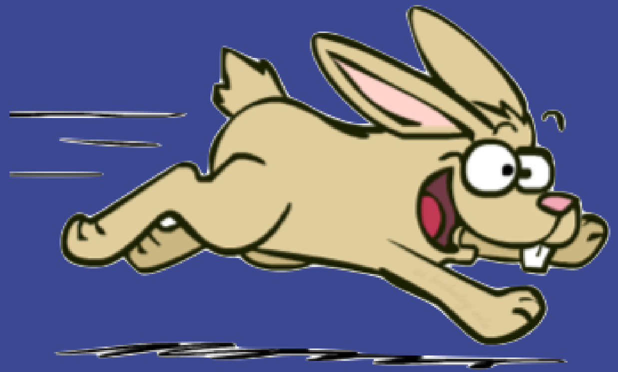
Maze: Deep Shallow Water

A. UCS
B. Greedy
C. A*
D. DFS



A* Recap

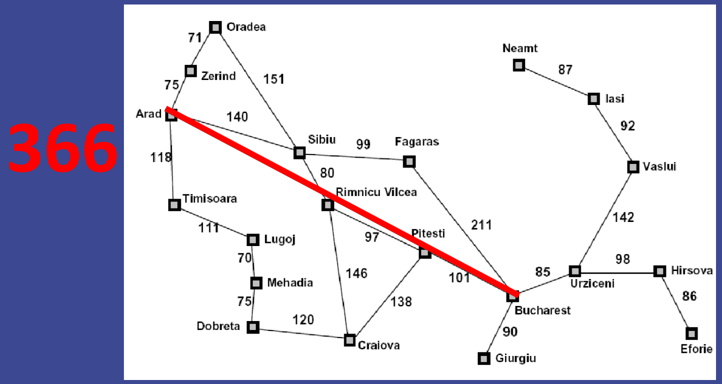
- A* uses both backward costs and (estimates of) forward costs
- A* is optimal with admissible and consistent heuristics
- Heuristic design is key

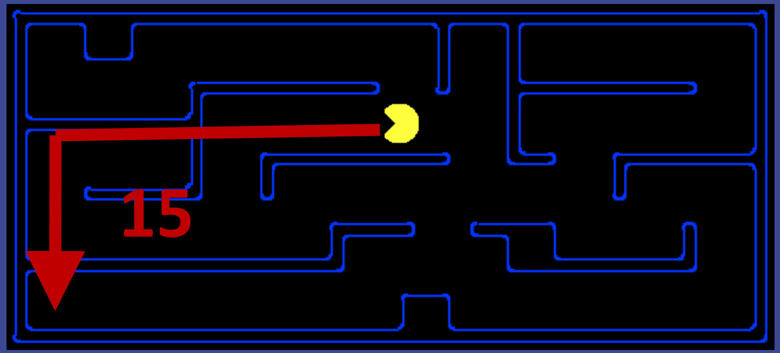




Creating Admissible Heuristics

Creating Admissible Heuristics

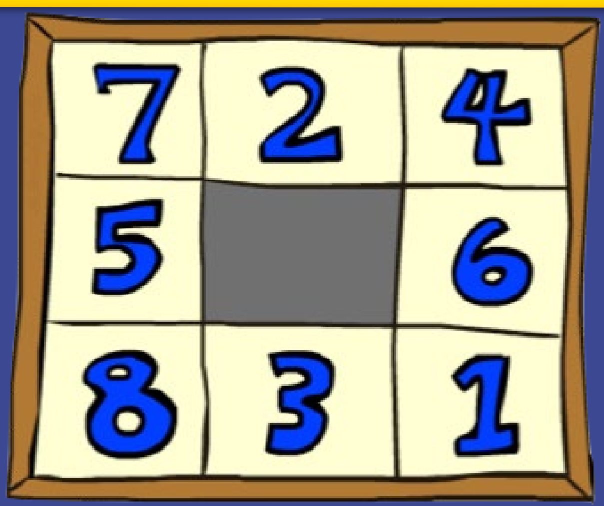
- Most of the work in solving hard search problems optimally is in coming up with admissible heuristics.
- Often, admissible heuristics are exact solutions to relaxed, easier problems.



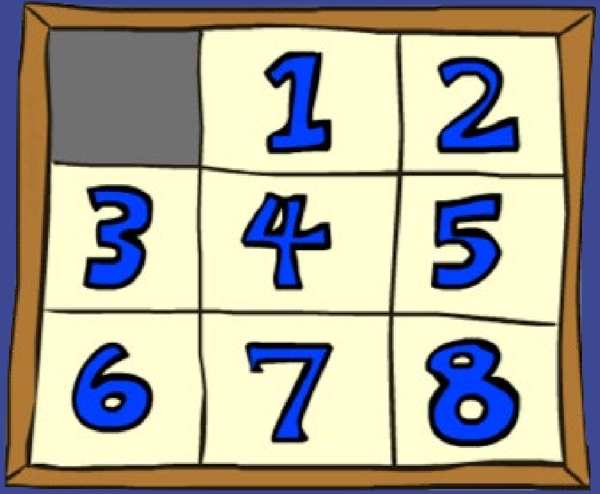


- Inadmissible heuristics are often useful too - suboptimal is sometimes ok.

Example: 8 Puzzle



Start State



Goal State

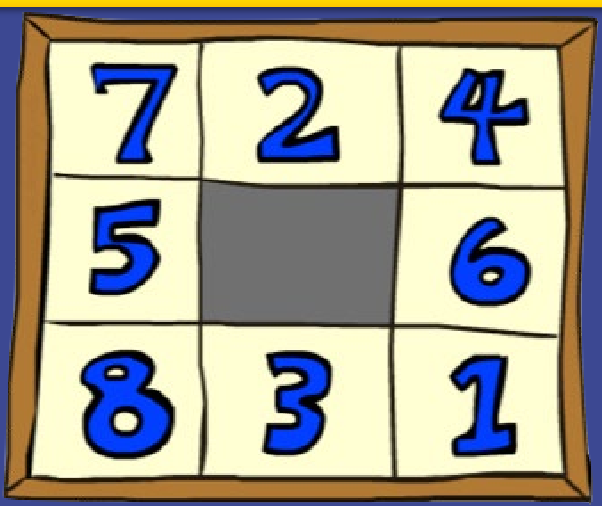
What are the states?

- permutations of 9 squares

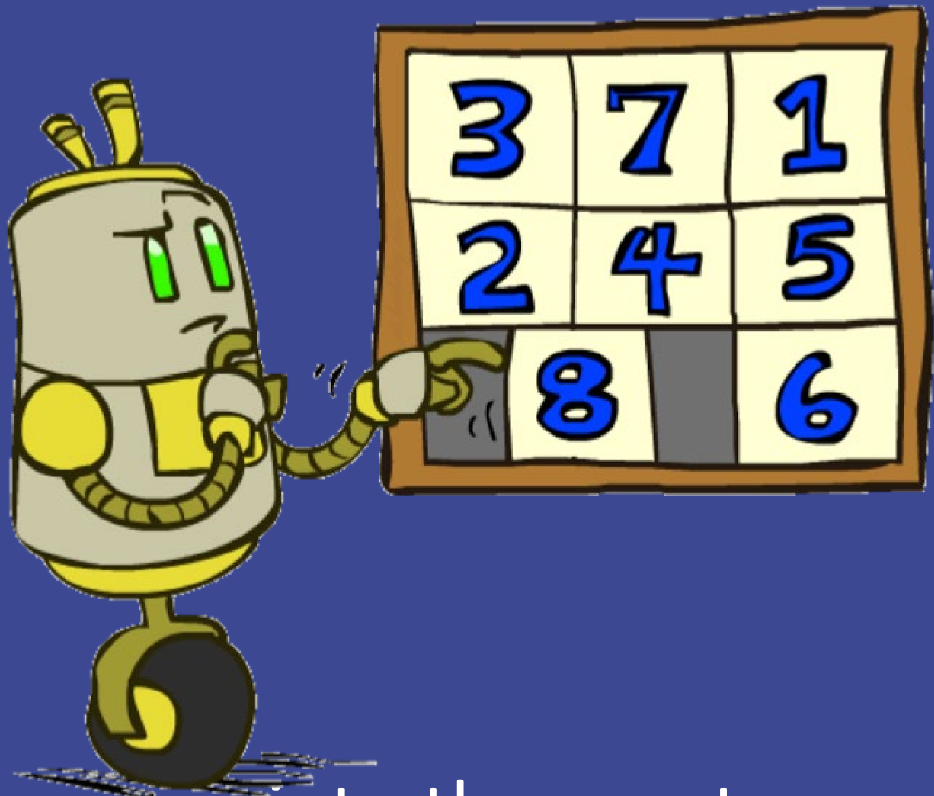
How many states?

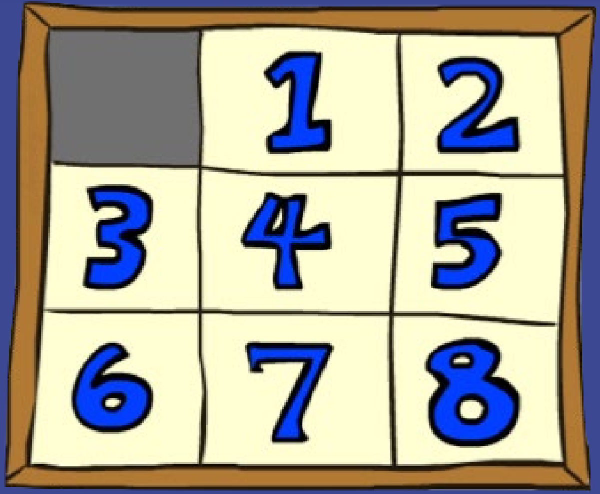
- 9!

Example: 8 Puzzle



Start State





Goal State

Actions?

- move a number square into the empty space / move empty

Costs?

- 1

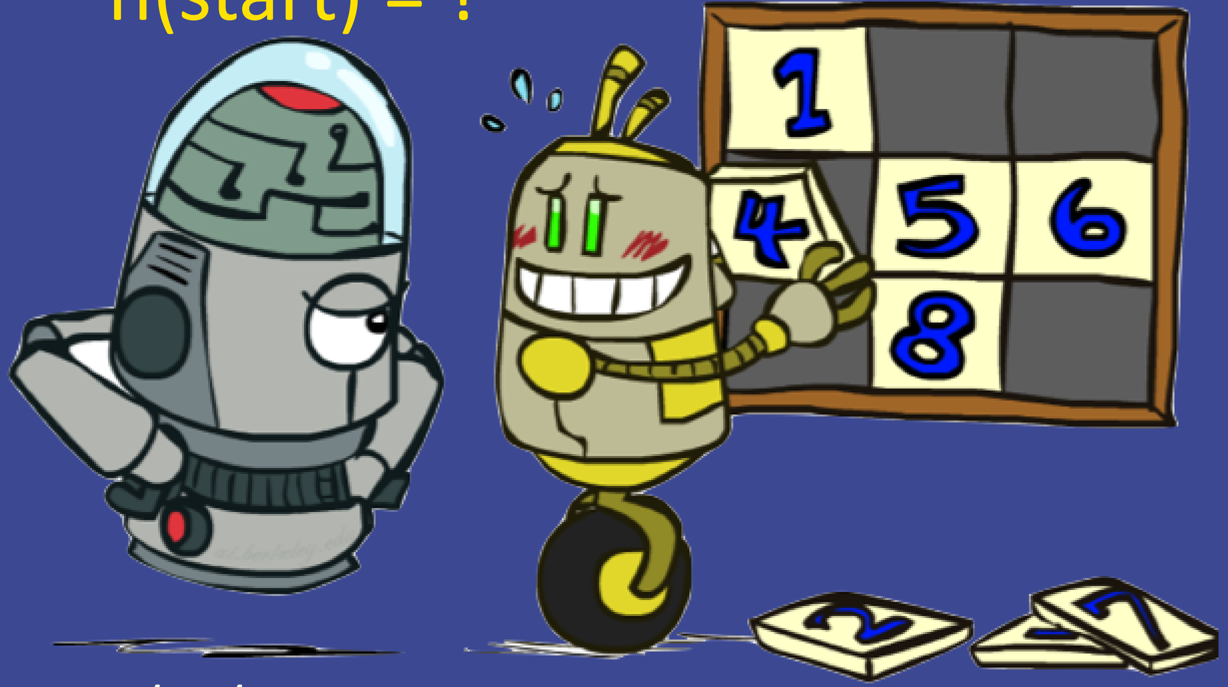
Heuristic 1: 8 Puzzle

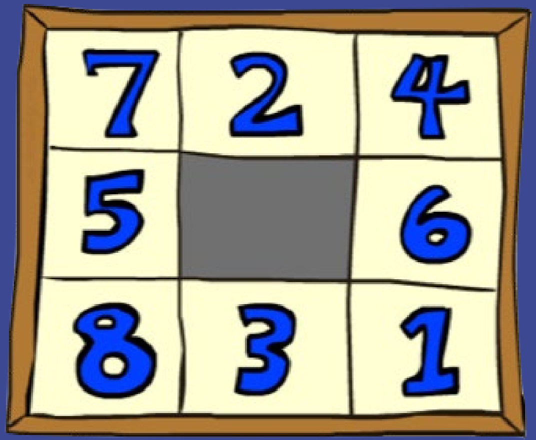
Heuristic?

- Number of tiles misplaced

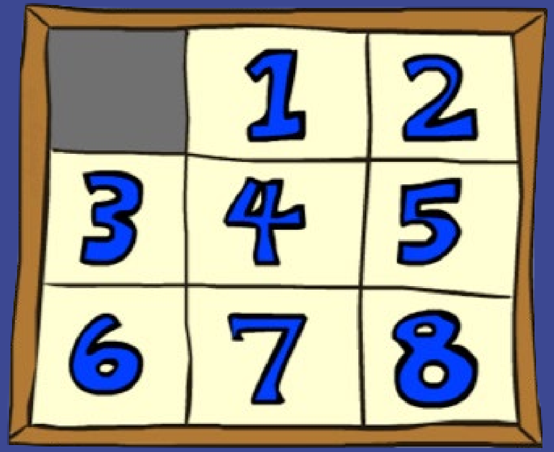
iClicker

h(start) = ?





Start State



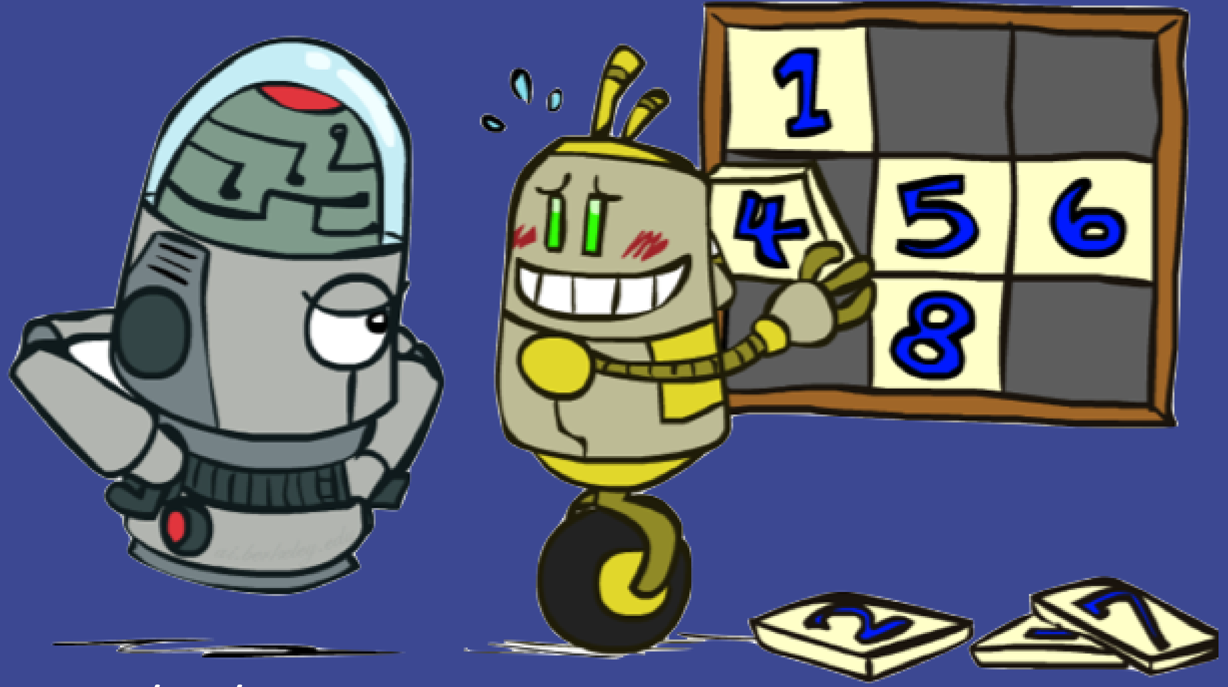
Goal State

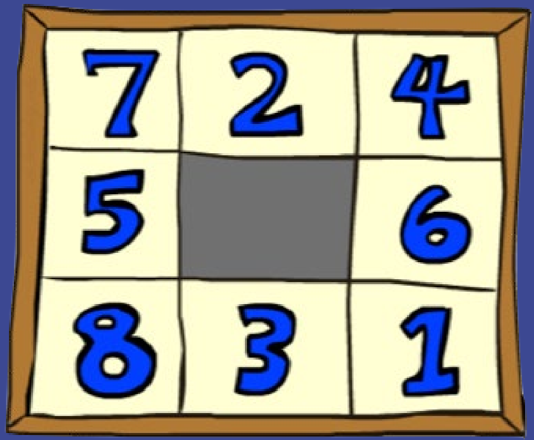
Heuristic 1: 8 Puzzle

Heuristic?

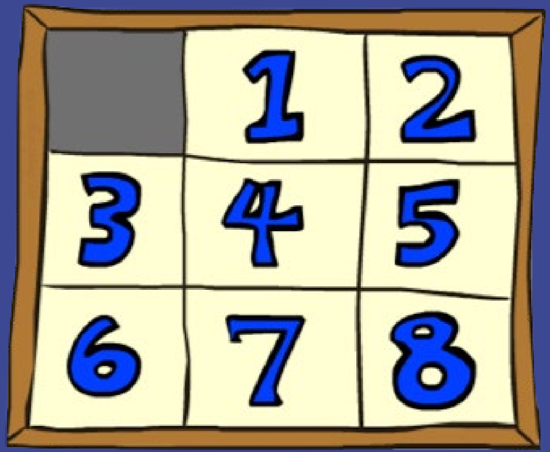
- Number of tiles misplaced

Why is it admissible?





Start State



Goal State

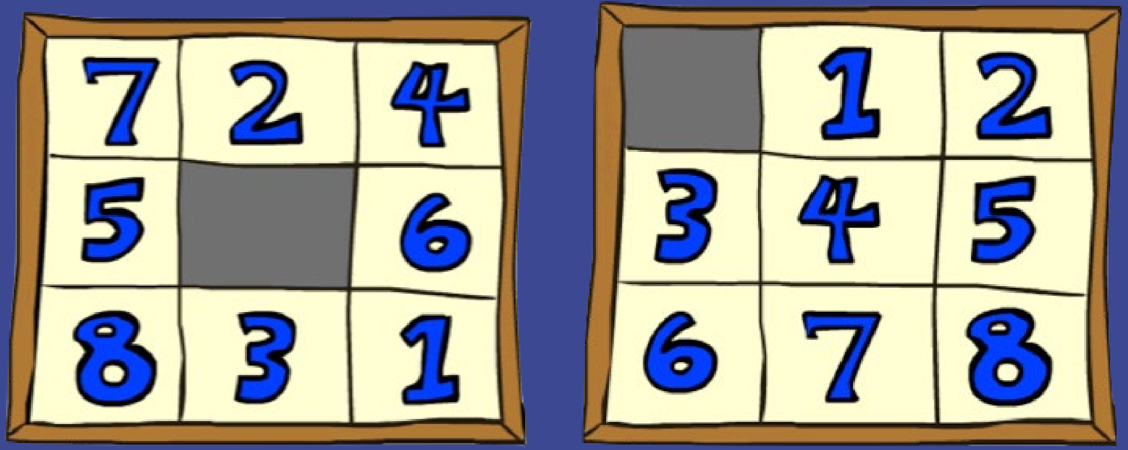
It is a relaxed-problem heuristic.

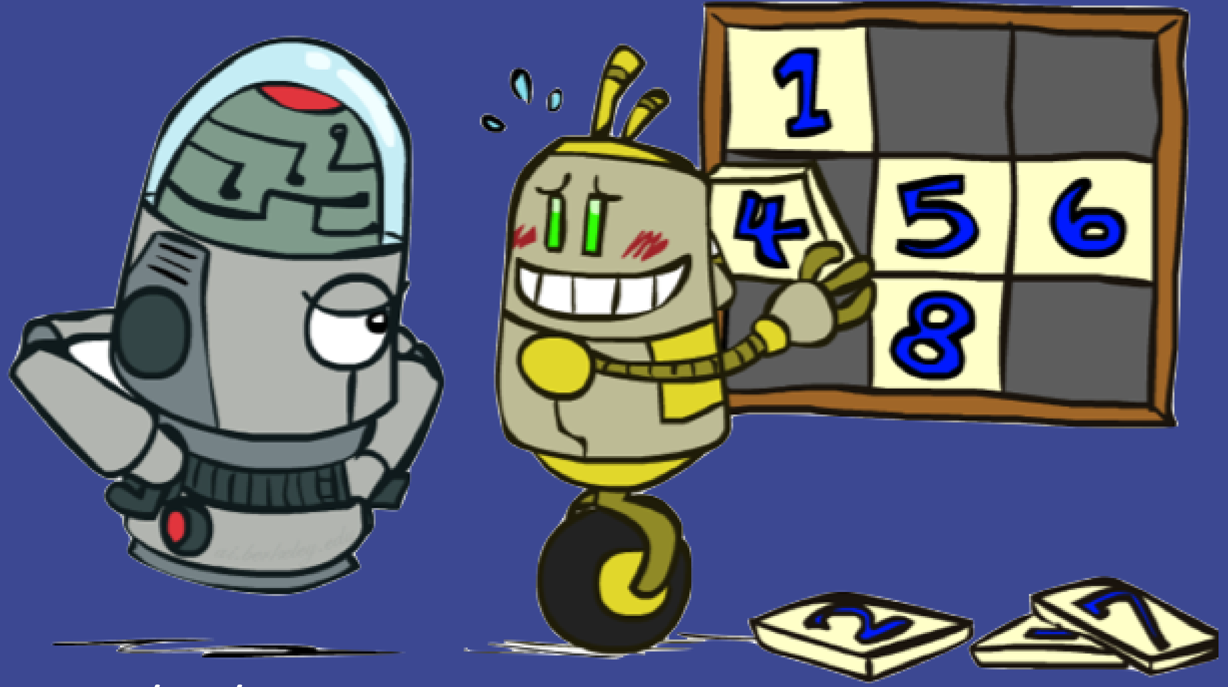
Heuristic 1: 8 Puzzle

Heuristic?

- Number of tiles misplaced

Why is it admissible?





	Average nodes expanded when the optimal path has...		
	...4 steps	...8 steps	...12 steps
UCS	112	6,300	3.6×10^6
TILES	13	39	227

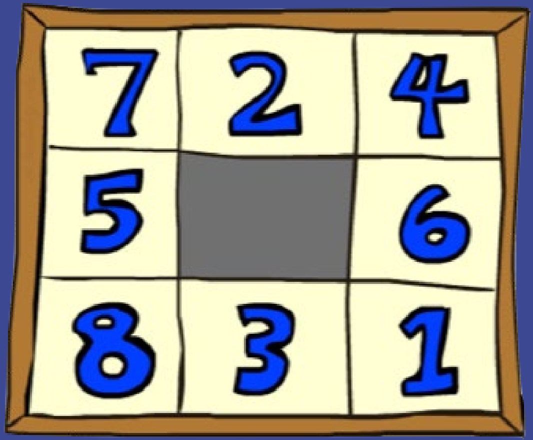
Statistics from Andrew Moore

Heuristic 2: 8 Puzzle

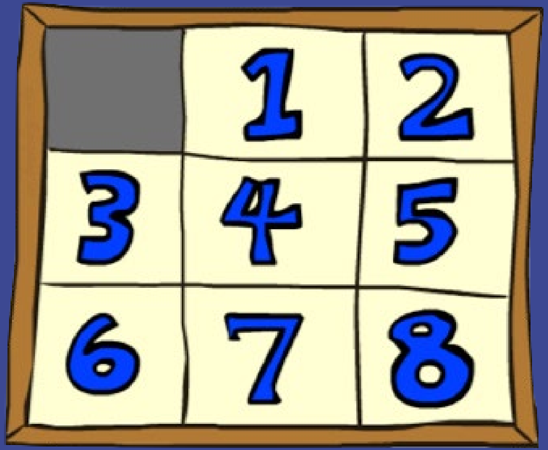
What if we had an easier 8-puzzle where any tile could slide any direction at any time, ignoring other tiles?

- Heuristic: total Manhattan distance to target position

Why is it admissible?



Start State



Goal State

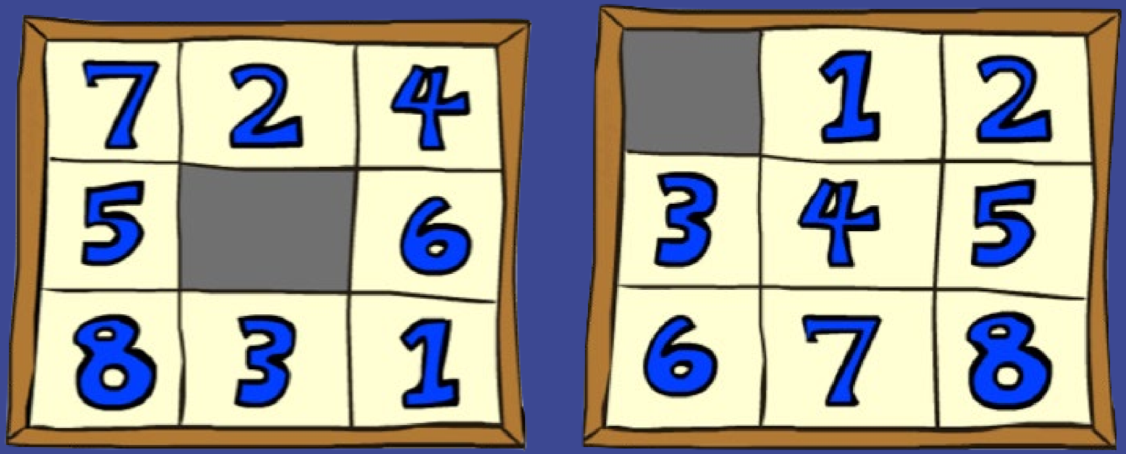
h(start) = ?

Heuristic 2: 8 Puzzle

What if we had an easier 8-puzzle where any tile could slide any direction at any time, ignoring other tiles?

- Total Manhattan distance to target position

Why is it admissible?



h(start) = 18

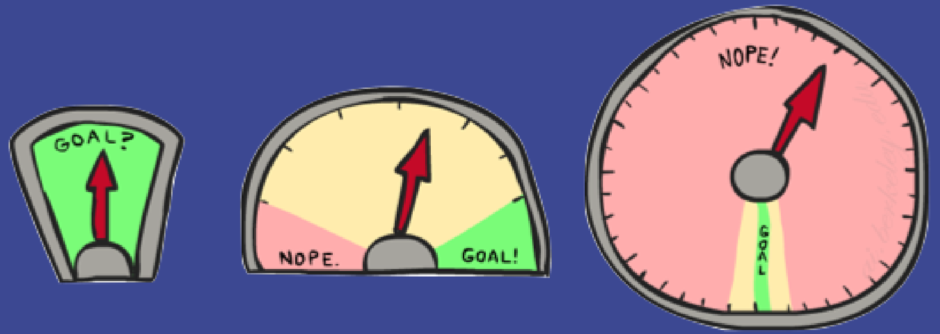
	Average nodes expanded when the optimal path has…		
	…4 steps	…8 steps	…12 steps
TILES	13	39	227
MANHATTAN	12	25	73

Statistics from Andrew Moore

Heuristics

How about using the actual cost as a heuristic?

- Would it be admissible?
- Would we save on nodes expanded?
- What's wrong with it?



With A*: a trade-off between quality of estimate and work per node

- As heuristics get closer to the true cost, we'll expand fewer nodes but usually do more work per node to compute the heuristic itself.

To do

- Homework 1: due at 5 PM
- Homework 2: Team assignment
  - Implement A* (similar to homework 1)
  - Design heuristics
  - Due September 23